

OpenFOAM Foundation v12 进阶教程

1. 网格生成工具

1.1 blockMesh: 结构化网格生成

1.1.1 文件结构与字典语法 blockMesh 是 OpenFOAM 中最基础且可靠的结构化网格生成工具，其配置文件 blockMeshDict 位于 constant/polyMesh/ 目录下。该文件采用 OpenFOAM 标准的字典格式，具有严格的层级结构和语法规则，以分号 ; 作为语句结束符，支持 C++ 风格的注释。

OpenFOAM v12 引入了增强的宏扩展功能，极大提升了参数化建模的灵活性：

指令	功能	示例
#eval	算术表达式求值	#eval{2*\$D} 计算两倍直径
#calc	复杂数学运算	#calc "degToRad(\$angle)" 角度转弧度
#if / #else / #endif	条件编译	切换二维/三维配置

单位系统通过 convertToMeters 关键字统一控制，所有坐标值在读取时乘以该系数。例如 convertToMeters 0.001 表示输入坐标单位为毫米。**量纲一致性检查**在解析阶段自动进行，任何单位不匹配都会导致明确的错误提示。

文件的基本框架包含标准 FoamFile 头结构，指定版本、格式、类别、位置和对象名称。v12 强化了字典解析的容错性，但对语法严格性要求不变——**CRLF 换行符**（Windows 风格）是跨平台协作中的常见陷阱，会导致神秘的“unexpected token”错误。

1.1.2 关键参数详解 **vertices 顶点定义**：顶点采用从零开始的整数索引，每个顶点由三维坐标 (x y z) 定义。顶点顺序必须遵循**右手定则**：当沿块边界遍历时，内部区域应始终位于左侧。对于复杂几何，建议采用系统化的参数化命名，通过 #eval 确保共享边界的坐标完全一致。

blocks 块拓扑结构：支持 hex（六面体，8 顶点）、wedge（楔形，6 顶点，轴对称）、prism（棱柱，6 顶点）等类型。每个块定义包含：

- 顶点索引列表（hex 需 8 个，按特定顺序）
- 网格划分数（nx ny nz）
- 膨胀比控制：simpleGrading (rx ry rz) 或 edgeGrading (12 条边独立控制)

膨胀比	效果	典型应用
> 1	向末端加密	壁面边界层（底部稀疏，壁面密集）
< 1	向起始端加密	自由表面区域（水面附近加密）
= 1	均匀分布	远场区域

edges 边定义：曲线边包括 arc（圆弧，需指定弧上点）、spline（样条曲线，多控制点）、polyLine（多段线）和 BSpline。圆弧的精度直接影响曲面几何保真度，圆环入水的环截面圆周至少需 8-10 个插值点。

patches 边界类型：边界类型的选择直接决定物理行为：

类型	物理意义	典型应用
wall	无滑移壁面	固体表面、环表面

Table 3 – continued

类型	物理意义	典型应用
patch	通用边界	入口、出口、远场
symmetry / symmetryPlane	对称面	减少计算域 (1/4 或 1/2 域)
cyclic / cyclicAMI	周期性边界	旋转机械、重复结构
empty	零厚度边界	二维问题的第三方向

mergePatchPairs 多块拼接：通过指定配对补丁名称实现多块网格的拓扑连接。关键要求是两侧补丁的顶点在容差范围内精确重合，建议通过 `#eval` 宏确保坐标一致性，而非手动输入数值。

1.1.3 典型示例：圆环入水计算域 以下展示三维圆环入水问题的参数化 `blockMeshDict` 框架：

```

/*-----*- C++ -*-----*\
| OpenFOAM: The Open Source CFD Toolbox - v12                                |
\*-----*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}
// *****

convertToMeters 1;

// === 几何参数 ===
D      0.050;      // 环直径 [m]
d      0.005;      // 环线径 [m]
H_drop 0.100;      // 下落高度 [m]
H_water 0.200;     // 水深 [m]
H_air   0.150;     // 空气域高度 [m]
W_domain #eval{5*D}; // 域宽度 = 5D

// === 派生尺寸 ===
R      #eval{0.5*D};      // 环半径
r_wire #eval{0.5*d};      // 线半径
R_outer #eval{R + 2*r_wire}; // 加密区外径

// === 网格参数 ===
n_radial 60;      // 径向单元数
n_vertical 80;     // 垂直方向单元数
n_azimuth 64;     // 周向单元数 (完整 360°)

vertices
(
    // 底部水域角点 (0-7)
    (-W_domain -W_domain 0)
    ( W_domain -W_domain 0)

```

```

( $W_domain $W_domain 0)
(-$W_domain $W_domain 0)
(-$W_domain -$W_domain -$H_water)
( $W_domain -$W_domain -$H_water)
( $W_domain $W_domain -$H_water)
(-$W_domain $W_domain -$H_water)

// 顶部空气域角点 (8-15)
(-$W_domain -$W_domain $H_air)
( $W_domain -$W_domain $H_air)
( $W_domain $W_domain $H_air)
(-$W_domain $W_domain $H_air)
// ... 对应下层顶点
);

blocks
(
    // 水域块: 底部稀疏, 向水面加密
    hex (0 1 2 3 4 5 6 7)
        ($n_radial $n_radial #eval{int(0.6*$n_vertical)})
        simpleGrading (1 1 0.3) // z 向: 底部 → 水面加密

    // 空气域块: 水面附近加密, 向上稀疏
    hex (0 1 2 3 8 9 10 11)
        ($n_radial $n_radial #eval{int(0.4*$n_vertical)})
        simpleGrading (1 1 3) // z 向: 水面 → 顶部拉伸
);

edges
(
    // 环几何将通过 snappyHexMesh 嵌入, 或在此定义弧边
);

boundary
(
    bottom
    {
        type    wall;
        faces   ((4 5 6 7));
    }

    atmosphere
    {
        type    patch;
        faces   ((8 9 10 11));
    }

    sides
    {
        type    patch;
        faces

```

```
(
    (0 4 7 3) (2 6 5 1)
    (1 5 4 0) (3 7 6 2)
    (0 8 11 3) (2 10 9 1)
    (1 9 8 0) (3 11 10 2)
);

}

// 对称面 (若使用 1/4 域)
symmetryX
{
    type    symmetry;
    faces   ((0 4 7 3) (0 8 11 3));
}
symmetryY
{
    type    symmetry;
    faces   ((0 1 5 4) (0 1 9 8));
}
);

mergePatchPairs();

// ***** //
```

该设计体现了**分层加密策略**：水域向水面压缩（grading 0.3），空气域向上拉伸（grading 3），在界面区域形成最优分辨率分布。

1.1.4 常见问题与解决

问题	症状	诊断	解决方案
顶点顺序错误	“negative volume cells” 或 “face area check failed”	checkMesh 检测负体积；ParaView 可视化法向方向	调整块顶点顺序，确保右手定则
CRLF 换行符	“unexpected token” 或行号错位	file blockMeshDict 显示 “CRLF line terminators”	dos2unix blockMeshDict 或 sed -i 's/\r\$//'
非凸块几何	“concave cells” 或 “face twist” 警告	可视化检查块形状	拆分为多个凸块，或用 snappyHexMesh 替代
单位换算错误	网格尺寸异常（过大/过小）	对比预期与实际边界框	核对 convertToMeters 设置

1.2 snappyHexMesh：复杂几何非结构化网格

1.2.1 字典结构与工作流程 snappyHexMesh 采用独特的**三阶段工作流程**，全部控制在 system/snappyHexMeshDict 中：

阶段	功能	关键输出	启用开关
castellatedMesh	背景网格切割与局部加密	阶梯状近似表面	castellatedMesh true;
snap	顶点投影与表面贴合	光滑贴合表面	snap true;
addLayers	壁面边界层添加	棱柱层网格	addLayers true;

工作流程始于 blockMesh 生成的背景六面体网格——该网格应足够粗以覆盖整个域，但需捕捉几何主要特征。随后导入 STL/OBJ 表面几何，通过递归细分（refinement）在表面附近生成细密网格。

1.2.2 表面几何处理 STL 预处理是网格质量的基础。必需步骤包括：

- **法向一致性**：所有三角形法向指向流体域外部
- **几何封闭**：无自由边、孔洞或自相交
- **特征尺寸**：面片尺寸与目标网格分辨率匹配

surfaceFeatureExtract 工具提取锐边特征，生成 .eMesh 文件：

```
surfaceFeatureExtract -includedAngle 150 constant/triSurface/ring.stl ring.eMesh
```

includedAngle 参数控制特征识别敏感度：过小则噪声过多，过大则遗漏关键边。圆环的光滑曲面推荐 **150°**，焊接凸起等细节区域可降至 **120°**。

snappyHexMeshDict 核心配置：

```
/*-----*- C++ -*------*\
| snappyHexMeshDict - 圆环入水网格生成                               |
\*-----*-/*
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    object       snappyHexMeshDict;
}
// * * * * *

castellatedMesh true;
snap           true;
addLayers      true;

geometry
{
    ring.stl
    {
        type      triSurfaceMesh;
        name      ring;
        regions
        {
            ring_surface { name ring_wall; }
        }
    }
}

// 局部加密区域：环周围圆柱
refinementCylinder
{
    type      searchableCylinder;
    point1    (0 0 -0.05);
    point2    (0 0 0.20);
}
```

```

        radius 0.08;
    }
};

castellatedMeshControls
{
    maxLocalCells      100000;
    maxGlobalCells     5000000;
    minRefinementCells 10;
    nCellsBetweenLevels 2;

    features
    (
        {
            file      "ring.eMesh";
            level      2;          // 特征边加密等级
        }
    );

    refinementSurfaces
    {
        ring_wall
        {
            level      (2 3); // (最小 最大) 表面加密
            patchInfo   { type wall; }
        }
    }

    refinementRegions
    {
        refinementCylinder
        {
            mode        inside;
            levels       ((0.02 4) (0.05 3) (0.10 2)); // 距离-等级对
        }
    }

    locationInMesh      (0 0 0.05); // 域内种子点
    allowFreeStandingZoneFaces true;
};

snapControls
{
    nSmoothPatch        3;
    tolerance            2.0;          // 投影容差 (局部特征尺寸倍数)
    nSolveIter          100;
    nRelaxIter           5;

    nFeatureSnapIter     10;          // 特征捕捉迭代
    implicitFeatureSnap  false;
    explicitFeatureSnap  true;        // 使用预定义 .eMesh

```

```

    // 质量控制
    minTetQuality      1e-15;
    maxNonOrtho        65;
};

addLayersControls
{
    relativeSizes      true;          // 相对局部单元尺寸

    // 层分布
    expansionRatio      1.2;          // 层间膨胀比
    finalLayerThickness 0.5;          // 最外层相对厚度
    minThickness        0.1;          // 最小绝对厚度

    // 特征处理
    featureAngle        60;           // 特征角阈值
    slipFeatureAngle    30;           // 滑移特征角

    // 迭代控制
    nGrow                0;
    nLayerIter           50;
    nRelaxedIter         5;           // 质量不满足时的松弛迭代

    // 质量阈值
    maxFaceThicknessRatio 0.5;
    maxThicknessToMedialRatio 0.3;
};

meshQualityControls
{
    maxNonOrtho          65;
    maxBoundarySkewness  20;
    maxInternalSkewness  4;
    maxConcave           80;
    minVol                1e-13;
    minTetQuality         1e-15;
    minArea               -1;
    minTwist              0.02;
    minDeterminant        0.001;
    minFaceWeight         0.02;
    minVolRatio           0.01;

    nSmoothScale          4;
    errorReduction        0.75;
};

writeFlags
(
    scalarLevels
    layerSets

```

```
        layerFields
    );

    mergeTolerance 1e-6;

    // ***** //
```

局部加密策略: refinementRegions 的 distance 模式实现从环表面向外的渐变加密，避免分辨率突变导致的数值问题。距离-等级对 (0.02 4) 表示 2cm 内使用等级 4（背景网格的 1/16），(0.05 3) 表示 2-5cm 使用等级 3，以此类推。

1.2.3 网格质量控制参数 snap 阶段的参数平衡是几何贴合精度与网格质量的核心矛盾：

参数	保守设置	激进设置	调优策略
tolerance	1.0	4.0	贴合不足时增大，发散时减小
nSmoothPatch	3	10	表面振荡时增加
nFeatureSnapIter	5	15	锐边保持不足时增加
maxNonOrtho	60°	75°	质量优先 vs 贴合优先

addLayers 阶段的失败率最高，系统性排查流程：

1. 检查 log.snappyHexMesh 的层添加统计（“Added X layers”）
2. 可视化 layerCoverage 场识别失败区域
3. 常见问题：几何间隙 < 层厚度、特征边处层折叠、表面曲率过大

1.2.4 并行网格生成 大规模网格生成的标准并行流程：

```
# 1. 生成背景网格
blockMesh

# 2. 域分解 (scotch 自动优化)
decomposePar

# 3. 并行运行 snappyHexMesh
mpirun -np 8 snappyHexMesh -parallel -overwrite

# 4. 网格重组 (可选)
reconstructParMesh -constant
```

关键注意事项：

- 背景网格分解后，各分区的 locationInMesh 必须唯一确定内部/外部
- scotch 方法产生不规则分区边界，可能略微影响表面贴合一致性
- v12 改进的并行一致性算法支持零单元处理器的重分布

1.2.5 常见问题与解决

问题	症状	根本原因	解决方案
表面穿透	snap 后表面”尖刺”或负体积	tolerance 过大或背景网格过粗	减小 tolerance 至 1.0-2.0; 增加基础加密等级
迭代不收敛	残差振荡或达到最大迭代	松弛因子冲突或质量阈值过严	增加 nRelaxIter; 放宽 maxNonOrtho 临时验证
边界层添加失败	“0 layers added”或厚度不均	几何间隙、特征角冲突、表面质量差	修复 STL; 调整 feature-Angle/slipFeatureAngle; 局部禁用层添加
特征边丢失	环截面圆化	explicitFeatureSnap 未启用或 .eMesh 精度不足	启用 explicitFeatureSnap; 减小 includedAngle 重新提取特征

1.3 网格质量检查与改进

1.3.1 checkMesh 输出解读 checkMesh 是网格质量评估的金标准，输出分为统计信息、拓扑检查、几何检查三个层次：

关键质量指标与阈值：

指标	定义	可接受	严重问题	影响
非正交性	面法向与连接向量夹角	< 70°	> 85°	扩散误差、收敛困难
偏斜度	面中心偏离程度	< 4	> 10	插值精度、局部极值
长宽比	最大/最小尺寸比	< 100	> 1000	各向异性误差
最小体积	单元体积	> 0	负体积	求解器崩溃

典型输出片段解读：

```
Mesh stats
  points:      1250423
  faces:      3657821
  cells:      1203456
  boundary patches: 6

Checking geometry...
  Max non-orthogonality = 65.3 degrees // 良好
  Max skewness = 2.8 // 优秀
  Max aspect ratio = 45.3 // 可接受
```

错误分级处理：

- **Failed**：必须修复（负体积、拓扑错误）
- **Severe**：强烈建议修复（非正交性 > 80°）
- **Warning**：评估后决定（边界层区域的长宽比）

1.3.2 质量改进策略 分级改进流程：

级别	方法	适用场景	成本
1	参数调优	单一指标略超阈值	低
2	refineMesh + topoSet	局部问题集中	中
3	表面重构 + 重新 snap	系统性表面质量问题	中高
4	退回 blockMesh 重新设计	拓扑根本性缺陷	高

topoSet 选择高非正交性单元的示例：

```
// system/topoSetDict
actions
(
    {
        name    badCells;
        type    cellSet;
        action  new;
        source  fieldToCell;
        sourceInfo
        {
            field  nonOrthoAngle;
            min    60;
            max    90;
        }
    }
);
```

随后使用 refineMesh 对这些单元进行各向同性加密，或结合 cellSet 在 snappyHexMesh 中排除问题区域。

2. interFoam 求解器与物理模型

2.1 文件结构与初始化

2.1.1 必需文件清单 interFoam 案例的完整文件结构：

目录	必需文件	物理意义	v12 注意事项
0/	p_rgh	修正压力 (p - gh)	推荐使用，改善静水压力处理
0/	U	速度场	三维矢量
0/	alpha.water	水相体积分数	命名需与 phases 一致
0/	k, epsilon/omega	湍流量	RAS/LES 时必需
constant/	transportProperties	相属性与界面物理	表面张力模型结构更新
constant/	turbulenceProperties/momentumTransport	湍流模型选择	v12 推荐 momentumTransport
constant/	g	重力加速度	统一量纲向量
constant/polyMesh/	网格文件	拓扑与几何	由 blockMesh/snappyHexMesh 生成
system/	controlDict	时间控制与 I/O	函数对象位置调整
system/	fvSchemes	离散格式	VOF 关键格式选择

Table 10 – continued

目录	必需文件	物理意义	v12 注意事项
system/	fvSolution	求解器参数	PIMPLE 算法控制
system/	setFieldsDict	非均匀初始场	溃坝、环初始位置等

⚠️ 版本差异: v12 中函数对象推荐置于 system/functions 文件, 而非 controlDict 内嵌, 以提高可维护性。

2.1.2 transportProperties 详解 transportProperties 是 interFoam 最核心的物理定义文件。v12 采用更模块化的结构:

```
/*-----*- C++ -*------*\
| transportProperties - 圆环入水两相流                                |
\*-----*-*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
    object       transportProperties;
}
// ***** //

phases (water air);

water
{
    transportModel  Newtonian;
    nu              1e-06;      // 运动粘度 [m²/s]
    rho            1000;      // 密度 [kg/m³]
}

air
{
    transportModel  Newtonian;
    nu              1.48e-05;
    rho            1.225;
}

// === 表面张力模型 ===
// $$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/2705.png}}}}$$$
⇨ 推荐: 显式模型定义 (v12)
sigma
{
    type          constant;
    sigma         0.072;      // N/m, 20°C 水-空气
}

// $$$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/docminer/emoji/
⇨ 26a0-fe0f.png}}}}$$$ 已废弃: 直接指定 sigma 常数值 (仍兼容, 但不推荐)
```

```
// sigma          0.072;

// === 界面压缩控制 ===
interfaceCompression
{
    cAlpha          1;          // 压缩系数, 0-4, 默认 1
}

// ***** //
```

表面张力模型类型对比:

类型	适用场景	关键参数
constant	标准等温问题	sigma 常数值
temperatureDependent	热毛细对流	sigma0, T0, dSigma/dT
coded	自定义依赖关系	运行时编译代码

对于圆环入水的 Worthington 射流研究，表面张力直接影响空腔闭合时间和喷泉高度，是与 Gekle & Gordillo (2009) 标度律对比的敏感参数。

2.1.3 turbulenceProperties 配置 湍流模型选择依据流动特征和计算资源:

```
/*-----*- C++ -*-----*\
| momentumTransport (v12 推荐) 或 turbulenceProperties |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
    object       momentumTransport;
}
// * * * * * //

simulationType RAS;          // laminar | RAS | LES

RAS
{
    model        kOmegaSST; // $$\vcenter{\hbox{\includegraphics[height=\ht\strutbox]{/usr/share/
    ↪ docminer/emoji/2705.png}}}}$$ 推荐: 多相流分离流
    turbulence    on;
    printCoeffs   on;

    // 近壁面处理
    wallFunction
    {
        k         kqRWallFunction;
        omega      omegaWallFunction;
        nut         nutkWallFunction;
```

```
    }
}

// ***** //
```

RAS 模型选择指南:

模型	优势	劣势	多相流适用性
kEpsilon	稳健、低成本	各向同性、近壁面差	远场充分湍流
kOmegaSST	近壁面精度高、分离流好	对入口敏感	✅ 推荐默认
realizableKE	可实现性约束	计算成本略高	强剪切旋转流
RNGkEpsilon	低雷诺数修正	参数调优复杂	过渡区流动

近壁面处理与 y+ 估计:

第一层网格高度计算公式:

$$h_1 = \frac{y^+ \cdot \nu}{u_\tau} \approx \frac{y^+ \cdot \nu}{0.05 \cdot U_{ref}}$$

以圆环入水典型参数为例: $U_{ring} = 2 \text{ m/s}$, $\nu_{water} = 10^{-6} \text{ m}^2/\text{s}$, 目标 $y^+ = 1$ (解析粘性子层):

- $h_1 \approx 10^{-5} \text{ m} = 10 \text{ }\mu\text{m}$
- 对应 snappyHexMesh 的 firstLayerThickness 约 0.002 (相对局部单元尺寸)

2.2 数值方法设置

2.2.1 fvSchemes 离散格式选择 fvSchemes 控制方程各项的离散方法, 直接影响精度、稳定性和计算成本:

```
/*-----*- C++ -*------*\
| fvSchemes - VOF 多相流离散格式 |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "system";
    object       fvSchemes;
}
// ***** //

ddtSchemes
{
    default      Euler;           // 一阶隐式, 稳定
    // 或: CrankNicolson 0.9;      // 二阶, 0.9 混合增强稳定
}

gradSchemes
{
    default      Gauss linear;
    grad(U)      cellLimited Gauss linear 1;  // 速度梯度限制
}
```

```

    grad(alpha.water) Gauss linear;                // 界面曲率计算
}

divSchemes
{
    default          none;                        // 强制显式指定

    // === VOF 关键：相分数对流 ===
    div(phi,alpha.water)  Gauss interfaceCompression vanLeer 1;
    div(phiRb,alpha.water) Gauss linear;          // 相对速度修正

    // 动量方程
    div(rhoPhi,U)        Gauss linearUpwind grad(U);

    // 湍流变量
    div(phi,k)           Gauss upwind;
    div(phi,omega)       Gauss upwind;

    // 粘性项
    div((nuEff*dev2(T(grad(U))))) Gauss linear;
}

laplacianSchemes
{
    default          Gauss linear corrected;      // 非正交修正
    // 高非正交网格: limited corrected 0.5
}

interpolationSchemes
{
    default          linear;
}

snGradSchemes
{
    default          corrected;
}

fluxRequired
{
    default          no;
    p_rgh;
    pcorr;
    alpha.water;    // 确保质量守恒
}

// *****

```

VOF 离散格式关键选择:

格式	类型	特点	适用项
vanLeer	TVD 限制器	二阶精度、有界、单调	div(phi,alpha) 首选
interfaceCompression	VOF 专用	人工压缩保持界面锐利	配合 cAlpha 使用
MUSCL	高阶重构	更锐利界面，成本较高	高精度需求
linearUpwind	二阶迎风格式	有界性较好	动量方程
limitedLinear	斜率限制器	可调限制参数	强剪切流动

2.2.2 fvSolution 求解器控制 PIMPLE 算法是 interFoam 的核心求解框架，融合 PISO 的压力-速度耦合与 SIMPLE 的迭代修正：

```
/*-----*- C++ -*------*\
| fvSolution - PIMPLE 算法与 VOF 特定参数                               |
\*-----*/
FoamFile
{
    version      2.0;
    format        ascii;
    class          dictionary;
    location       "system";
    object         fvSolution;
}
// *****

solvers
{
    // === 压力方程：GAMG 多重网格 ===
    "p_rgh.*"
    {
        solver          GAMG;
        tolerance        1e-07;
        relTol           0.01;
        smoother         GaussSeidel;
        nPreSweeps        0;
        nPostSweeps       2;
        cacheAgglomeration on;
        agglomerator      faceAreaPair;
        nCellsInCoarsestLevel 100;
        mergeLevels       1;
    }

    // === 速度方程：平滑求解器 ===
    U
    {
        solver          smoothSolver;
        smoother         symGaussSeidel;
        tolerance        1e-08;
        relTol           0.1;
        nSweeps          1;
    }
}
```

```

// === 湍流变量 ===
"(k|omega|epsilon)"
{
    solver          smoothSolver;
    smoother        symGaussSeidel;
    tolerance        1e-08;
    relTol           0.1;
}

// === VOF 方程 ===
alpha.water
{
    nAlphaCorr       2;           // 修正迭代次数
    nAlphaSubCycles  2;           // 子循环，增强稳定性
    cAlpha           1;           // 界面压缩系数

    MULESCorr        yes;         // 有界性修正
    nLimiterIter      5;           // 限制器迭代

    solver          smoothSolver;
    smoother        symGaussSeidel;
    tolerance        1e-08;
    relTol           0;
}
}

PIMPLE
{
    momentumPredictor  yes;         // 动量预测步
    nOuterCorrectors    2;           // 外送代（瞬态 1-3，复杂流动 3-5）
    nCorrectors         2;           // 压力-速度耦合修正
    nNonOrthogonalCorrectors 1;      // 非正交修正（高非正交网格 2-3）

    // === 残差控制（可选，动态收敛） ===
    residualControl
    {
        U              1e-05;
        p_rgh          5e-04;
        "(k|omega|epsilon)" 1e-05;
    }
}

relaxationFactors
{
    fields
    {
        p_rgh          0.3;         // 压力欠松弛
    }
    equations
    {
        U              0.7;
    }
}

```



```
        "(k|omega|epsilon)" 0.7;
        "alpha.*"          1.0;          // VOF 不建议松弛
    }
}

// ***** //
```

PIMPLE 参数调优策略:

参数	保守设置	激进设置	适用场景
nOuterCorrectors	1	5	强耦合问题、大时间步
nCorrectors	2	4	压力收敛困难时增加
nNonOrthogonalCorrectors	0	3	非正交性 > 70° 时必需
cAlpha	0.5	4	界面锐利度 vs 稳定性权衡

2.2.3 稳定性调优策略 库朗数控制是 VOF 稳定性的首要保障:

```
// system/controlDict 片段
adjustTimeStep yes;
maxCo          1;          // 最大对流库朗数
maxAlphaCo     0.5;        // 界面库朗数 (更严格)
maxDeltaT      1e-3;       // 物理时间步上限
```

初始场平滑策略:

- 使用 setFields 的渐进过渡, 避免阶梯式 alpha 分布
- 启用 funkySetFields 实现误差函数型界面厚度
- 初始阶段临时降低 maxCo 至 0.1-0.2

数值发散早期诊断:

症状	可能原因	应急措施
bounding alpha	有界性违反	增加 nAlphaSubCycles, 降低 cAlpha
压力求解不收敛	网格质量差或时间步过大	增加 nNonOrthogonalCorrectors, 降低 maxCo
速度场 nan	边界条件冲突	检查 p_rgh 与 U 边界兼容性

2.3 自定义物理模型 (入门)

2.3.1 coded 机制原理 coded 机制是 OpenFOAM v12 实现用户自定义的核心特性, 基于 LLVM/Clang 运行时编译:

特性	说明
工作流程	字典代码 → 临时 C++ 源文件 → 动态编译 → 共享库加载
缓存位置	dynamicCode/ 目录, SHA1 哈希避免重复编译
首次延迟	10-30 秒 (编译时间)

Table 16 – continued

特性	说明
后续运行	直接加载缓存库，无延迟
调试方式	手动 <code>wmake</code> 或查看 <code>compile.log</code>

2.3.2 最小化示例：曲率相关表面张力模型 实现 $\sigma = (1 + K)$ 的曲率修正模型，研究曲率对喷泉高度的影响：

```
// constant/transportProperties 片段
sigma
{
    type    coded;
    name    curvatureDependentSigma;

    codeOptions
    #{
        -I$(LIB_SRC)/transportModels/interfaceProperties/lnInclude
        -I$(LIB_SRC)/finiteVolume/lnInclude
    #};

    codeLibs
    #{
        -linterfaceProperties
        -lfiniteVolume
    #};

    codeInclude
    #{
        #include "fvcGrad.H"
        #include "fvcDiv.H"
    #};

    code
    #{
        // 获取必要场
        const fvMesh& mesh = this->mesh();
        const volScalarField& alpha = mesh.lookupObject<volScalarField>("alpha.water");

        // 基础参数
        const dimensionedScalar sigma0("sigma0", dimForce/dimLength, 0.072);
        const dimensionedScalar beta("beta", dimLength, 1e-4); // 可调系数

        // 计算界面曲率（标准 VOF 方法）
        const volVectorField gradAlpha = fvc::grad(alpha);
        const volScalarField magGradAlpha = mag(gradAlpha) + SMALL;
        const volVectorField n = gradAlpha / magGradAlpha; // 单位法向
        const volScalarField K = -fvc::div(n); // 曲率（凸为正）

        // 曲率相关表面张力：高曲率区域  $\sigma$  降低（不稳定效应）
        return sigma0 * (1.0 - beta * mag(K));
    #};
}
```

```
}
}
```

编译测试与验证：

- 1. 首次运行自动触发编译，检查 dynamicCode/curvatureDependentSigma/compile.log
- 2. 对比常数 与曲率相关模型的空腔形态差异
- 3. 定量分析：喷泉高度、闭合时间、界面长度演化的敏感性

2.3.3 扩展应用：自定义粘度模型 剪切稀化流体的 Power-law 模型框架：

```
transportModel    coded;
codedName          powerLawViscosity;

code
#{
    const volVectorField& U = mesh().lookupObject<volVectorField>("U");
    const volScalarField strainRate = sqrt(2.0) * mag(symm(fvc::grad(U)));

    const dimensionedScalar k("k", dimDynamicViscosity, 0.01); // 稠度系数
    const scalar n = 0.7; // 幂指数 (<1 剪切稀化)

    return k * pow(strainRate + dimensionedScalar("SMALL", dimless/dimTime, 1e-10), n - 1.0) / rho_;
#};
```

与湍流模型耦合：确保自定义 nu 与 nut 正确叠加为 nuEff = nu + nut，RANS 模型自动处理。

3. 并行计算与域分解

3.1 decomposePar 域分解

3.1.1 分解方法选择

方法	算法	最佳适用	负载均衡	通信开销
scotch	图划分 (Scotch/METIS)	复杂几何、非结构化网格	优秀	中等
simple	坐标方向切割	规则矩形域	良好	较低
hierarchical	多级递归分解	大规模并行 (>1000 核)	可调	可控
ptscotch	并行图划分	超大规模	优秀	中等

圆环入水推荐：scotch 自动优化，或 hierarchical 配合 order xyz 控制切割顺序。

3.1.2 decomposeParDict 详解

```
/*-----*- C++ -*-*/
| decomposeParDict - 域分解配置 |
\*-----*/
FoamFile
{
```

```
version      2.0;
format       ascii;
class        dictionary;
object       decomposeParDict;
}
// ***** //

numberOfSubdomains 32;           // 与 mpirun -np 严格匹配

method        scotch;           // 或 simple, hierarchical

scotchCoeffs
{
    writeGraph    yes;           // 输出划分统计
    // strategy    "balance=gain,level=5"; // 高级优化
}

// 保留关键边界完整性
preservePatches
(
    atmosphere
    "inlet.%"
    "outlet.%"
);

// v12 新特性：区域特定分解（多区域案例）
// regions { fluid { ... } solid { ... } }

// ***** //
```

3.2 并行运行与监控

3.2.1 mpirun 启动命令

```
# 标准并行运行
mpirun -np 32 interFoam -parallel > log.interFoam 2>&1

# 集群提交 (SLURM)
#SBATCH --ntasks=32
mpirun interFoam -parallel

# 绑定优化
mpirun -np 32 --bind-to core --map-by ppr:4:node interFoam -parallel
```

3.2.2 运行监控与故障排查

错误	诊断	解决
number of decomposed domains does not match	numberOfSubdomains -np	统一修改

Table 18 – continued

错误	诊断	解决
cannot find file "processorX/..."	分解数据缺失	重新 decomposePar
某进程残差异异常高	子域网格质量差	检查该进程 checkMesh, 调整 preservePatches
计算缓慢、CPU 占用低	I/O 瓶颈或负载不均	启用 collated 格式, 优化分解策略

3.3 结果重组与一致性验证

3.3.1 reconstructPar 使用

```
reconstructPar          # 全部时间步
reconstructPar -time '0.1:0.5' # 范围
reconstructPar -latestTime    # 仅最新
reconstructPar -fields "(U p_rgh alpha.water)" # 指定场
```

并行后处理替代: paraFoam -parallel 直接加载分解数据, 避免重组开销。

3.3.2 并行-串行结果一致性 数值差异来源: 浮点求和顺序、迭代历史、随机数种子。验证准则: 关键物理量(喷泉高度、空腔深度)相对差异 < 1% 视为一致。

4. 后处理与数据提取

4.1 ParaView 可视化

4.1.1 案例加载与基础操作

```
touch case.foam      # 创建标识文件
paraFoam              # 或 paraview case.foam
```

4.1.2 相界面与流场可视化

操作	滤镜/设置	关键参数
相界面提取	Contour	alpha.water = 0.5
界面锐化	Smooth + Decimate	迭代 2-3 次
速度矢量	Glyph	Mask Points 控制密度
流线/迹线	Stream Tracer / Particle Tracer	种子点分布
涡结构	Calculator 计算 Q 准则	$Q = 0.5 * (\Omega ^2 - S ^2)$

4.1.3 高质量输出与动画

```
# ParaView 导出序列帧后, FFmpeg 编码
ffmpeg -framerate 30 -i frame_%04d.png \
-c:v libx264 -preset slow -crf 18 \
-pix_fmt yuv420p ringFountain.mp4
```

4.2 数据提取工具

4.2.1 sample 工具

```
// system/sampleDict
type            sets;
libs            (sampling);

setFormat       csv;

sets
{
    centerline
    {
        type     uniform;
        axis     y;
        start    (0 0 0);
        end      (0 0.5 0);
        nPoints  200;
    }
}

fields          (p_rgh U alpha.water);
```

4.2.2 函数对象 (functionObjects)

```
// system/functions 或 controlDict 内嵌
probes
{
    type            probes;
    libs            (sampling);

    writeControl     timeStep;
    writeInterval    1;

    probeLocations
    (
        (0 0 0)      // 对称轴原点
        (0 0.05 0)   // 预期喷泉区域
        (0 0.10 0)
        (0 0.20 0)
    );

    fields           (alpha.water U p_rgh);
}
```

4.3 Python/Shell 数据处理脚本

4.3.1 数据读取与清洗

```
#!/usr/bin/env python3
"""
```

OpenFOAM 场数据读取与喷泉高度提取

```
"""

import numpy as np
from pathlib import Path
import re

def read_foam_field(filepath):
    """ 读取 OpenFOAM ASCII 场文件, 返回 (header, data) """
    with open(filepath, 'r') as f:
        content = f.read()

    # 解析 FoamFile 头
    header_match = re.search(r'FoamFile\s*\{([^\}]+\)}', content, re.DOTALL)
    header = {}
    for line in header_match.group(1).strip().split('\n'):
        if ';' in line and '//' not in line.split(';')[0]:
            key, val = line.split(';')[0].strip().split(None, 1)
            header[key] = val.strip()

    # 提取 internalField
    internal_match = re.search(
        r'internalField\s+(uniform|nonuniform)\s+([^\;]+);',
        content
    )
    field_type, field_data = internal_match.groups()

    if field_type == 'uniform':
        values = [float(x) for x in re.findall(r'[-+]?\d*\.\d+?(:[eE][-+]?\d+)?', field_data)]
        data = np.array(values)
    else:
        # 解析 nonuniform List
        list_match = re.search(r'List<[^>+>\s*\{([^\}]+\)}', field_data, re.DOTALL)
        items = list_match.group(1).strip().split('\n')
        data = []
        for item in items:
            item = item.strip()
            if item.startswith('(') and item.endswith(')'):
                vec = [float(x) for x in re.findall(r'[-+]?\d*\.\d+?(:[eE][-+]?\d+)?', item)]
                data.append(vec)
            else:
                data.append(float(item))
        data = np.array(data)

    return header, data

def get_cell_centers(poly_mesh_path):
    """ 从 polyMesh 重建单元中心坐标 (简化版) """
    points_file = poly_mesh_path / 'points'
```

```

faces_file = poly_mesh_path / 'faces'
owner_file = poly_mesh_path / 'owner'

# 读取 points
_, points = read_foam_field(points_file)

# 简化: 假设均匀网格, 实际需完整拓扑重建
# 生产代码应使用 PyFoam 或 foamfile 库

return points # 占位返回

def extract_fountain_height(case_path, alpha_threshold=0.5):
    """
    从所有时间步提取喷泉最大高度

    Returns:
        times: 时间数组
        heights: 最大高度数组
        positions: 位置数组 (x, y, z)
    """
    case = Path(case_path)
    times = []
    heights = []
    positions = []

    # 遍历时间目录
    for time_dir in sorted(case.glob('[0-9]*')):
        if not time_dir.is_dir():
            continue

        try:
            t = float(time_dir.name)
        except ValueError:
            continue

        alpha_file = time_dir / 'alpha.water'
        if not alpha_file.exists():
            continue

        # 读取 alpha 场 (需配合网格坐标)
        _, alpha = read_foam_field(alpha_file)

        # 界面单元识别
        interface_mask = (alpha > 0.01) & (alpha < 0.99)
        if not np.any(interface_mask):
            continue

        # 获取单元中心 (简化, 实际需完整实现)
        # cell_centers = get_cell_centers(case / 'constant/polyMesh')
        # interface_centers = cell_centers[interface_mask]

```



```

# 简化: 假设对称轴上, 取  $\alpha > \text{threshold}$  的最大索引
# 生产代码应使用 probe 数据或完整场重建

# 占位: 从 probe 文件读取
probe_file = case / 'postProcessing' / 'probes' / '0' / 'alpha.water'
if probe_file.exists():
    probe_data = np.loadtxt(probe_file)
    # 假设最后一列是时间, 前面是各 probe 的  $\alpha$ 
    # 找到  $\alpha > 0.5$  的最大 y 位置
    pass # 具体实现依赖 probe 配置

return np.array(times), np.array(heights), np.array(positions)

if __name__ == '__main__':
    import matplotlib.pyplot as plt

    case_path = './ringFountain_case'
    times, heights, positions = extract_fountain_height(case_path)

    # 无量纲化:  $h/D$  vs  $Fr$ 
    D = 0.05 # 环直径
    g = 9.81

    # 假设恒定下落速度 (实际应从 U 场提取或已知条件)
    U = 2.0 # m/s
    Fr = U / np.sqrt(g * D)

    h_norm = heights / D

    plt.figure(figsize=(8, 6))
    plt.semilogy(Fr * np.ones_like(h_norm), h_norm, 'bo', label='Simulation')
    plt.xlabel('Froude number  $Fr = U/\sqrt{gD}$ ')
    plt.ylabel('Normalized fountain height  $h_{\max}/D$ ')
    plt.title('Ring Fountain: Simulation vs Gekle & Gordillo (2009)')
    plt.legend()
    plt.grid(True)
    plt.savefig('fountain_height_validation.png', dpi=300)

```

4.3.2 特征量自动提取 喷泉高度识别算法 (基于 probes 数据):

1. 读取 postProcessing/probes/0/alpha.water 时间序列
2. 对每个时间步, 找到所有 probes 中 $\alpha_{\text{water}} > 0.5$ 的最大 y 坐标
3. 记录全局最大高度及其出现时刻
4. 输出 $h_{\max}(t)$ 时间序列, 识别绝对最大值

5. 理论验证与脚本化分析

5.1 圆环入水 (Ring Fountain) 物理模型

5.1.1 问题描述与无量纲参数 圆环入水涉及复杂的多相流物理：环的惯性驱动液体向下形成空腔，空腔壁面不稳定性导致液体向上汇聚，最终在对称轴形成 Worthington 射流。

无量纲数	定义	物理意义	典型范围
Froude 数 Fr	$U/\sqrt{gD}$	惯性/重力	1 -100
Weber 数 We	$\rho U^2 D/\sigma$	惯性/表面张力	10^2 -10
Bond 数 Bo	$\rho g d^2/\sigma$	重力/表面张力 (基于线径)	0.1 -10
环厚比 η	d/D	几何细长比	0.01 -0.1

5.1.2 Gekle & Gordillo (2009) 标度律 核心标度关系：

$$\frac{h_{max}}{D} = f(Fr, We, Bo, \eta, \delta_H, \delta_W, \Gamma)$$

在 $We \gg 1$ 、 $Bo \sim 1$ 条件下，Froude 数为主导参数，简化为幂律形式：

$$\frac{h_{max}}{D} \approx A \cdot Fr^B$$

典型拟合值： $A \approx 0.5$ ， $B \approx 0.8$ (具体系数依赖 B_Fr_table.csv、Ac_Fr_table.csv 中的参数映射)。

5.2 模拟-理论对比流程

5.2.1 参数扫描自动化

```
#!/bin/bash
# parametric_study.sh: 圆环入水 Froude 数扫描

BASE_CASE="ringFountain_base"
DIAMETERS=(0.02 0.03 0.04 0.05 0.06) # m
VELOCITIES=(1.0 1.5 2.0 2.5 3.0 4.0) # m/s

for D in "${DIAMETERS[@]}; do
    for U in "${VELOCITIES[@]}; do

        # 计算无量纲参数
        Fr=$(echo "scale=4; $U / sqrt(9.81 * $D)" | bc)
        We=$(echo "scale=4; 1000 * $U^2 * $D / 0.072" | bc)

        CASE_NAME="D${D}_U${U}_Fr$(printf %.2f $Fr)"
        echo "Creating: $CASE_NAME (Fr=$Fr, We=$We)"

        cp -r "$BASE_CASE" "$CASE_NAME"
        cd "$CASE_NAME"

        # 参数化修改
        foamDictionary constant/polyMesh/blockMeshDict \
            -entry convertToMeters -set "1" # 确保单位一致
```

```

# 修改边界条件：入口速度
foamDictionary 0/U \
    -entry boundaryField/inlet/value \
    -set "uniform (0 0 -$U)"

# 运行流程
blockMesh > log.blockMesh 2>&1
decomposePar > log.decomposePar 2>&1
mpirun -np 8 interFoam -parallel > log.interFoam 2>&1

# 后处理提取
python3 ../extract_fountain_height.py

cd ..
done
done

# 汇总所有结果
python3 ../aggregate_results.py

```

5.2.2 喷泉高度提取算法 基于 probes 或 sample 数据的完整实现：

```

def extract_fountain_height_from_probes(case_path, probe_names=None):
    """
    从 probes 输出提取喷泉高度时间序列

    probe_names: 按 y 坐标排序的 probe 名称列表
    """
    post_path = Path(case_path) / 'postProcessing' / 'probes' / '0'

    # 读取 alpha.water probe 数据
    alpha_file = post_path / 'alpha.water'
    data = np.loadtxt(alpha_file, comments='#')

    times = data[:, 0]
    # 假设列顺序对应 probe 位置, 按 y 坐标递增

    # 对每个时间步, 找到 alpha > 0.5 的最高位置
    heights = []
    for i, t in enumerate(times):
        alpha_row = data[i, 1:] # 跳过时间列
        # 找到第一个 alpha < 0.5 的索引 (从顶部向下搜索)
        # 或找到 alpha > 0.5 的最大索引
        liquid_probes = np.where(alpha_row > 0.5)[0]
        if len(liquid_probes) > 0:
            max_idx = liquid_probes[-1] # 最高液相位置
            # 从 probe 配置获取 y 坐标
            height = probe_y_coords[max_idx]
            heights.append(height)
        else:

```

```

        heights.append(0) # 无液相

    return times, np.array(heights)

```

5.2.3 无量纲化与曲线拟合

```

import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

def power_law(Fr, A, B):
    """ 标度律拟合函数 """
    return A * Fr ** B

# 加载模拟数据
sim_data = np.loadtxt('simulation_results.csv', delimiter=',', skiprows=1)
Fr_sim = sim_data[:, 0] # Froude 数
h_norm_sim = sim_data[:, 1] # h_max/D

# 加载理论数据 (Gekle & Gordillo 2009)
theory_data = np.loadtxt('B_Fr_table.csv', delimiter=',', skiprows=1)
Fr_theory = theory_data[:, 0]
h_norm_theory = theory_data[:, 1]

# 拟合模拟数据
popt, pcov = curve_fit(power_law, Fr_sim, h_norm_sim, p0=[0.5, 0.8])
A_fit, B_fit = popt
A_err, B_err = np.sqrt(np.diag(pcov))

print(f"Fitted: h_max/D = ({A_fit:.3f} ± {A_err:.3f}) * Fr^{({B_fit:.3f} ± {B_err:.3f})}")

# 绘图
fig, ax = plt.subplots(figsize=(10, 7))

ax.loglog(Fr_theory, h_norm_theory, 'k-', linewidth=2, label='Gekle & Gordillo (2009)')
ax.loglog(Fr_sim, h_norm_sim, 'ro', markersize=8, alpha=0.7, label='Simulation')
ax.loglog(Fr_sim, power_law(Fr_sim, *popt), 'b--', linewidth=1.5,
          label=f'Fit: $h_{\{max\}}/D = {A_fit:.2f} Fr^{{({B_fit:.2f})}}$')

ax.set_xlabel('Froude number $Fr = U/\sqrt{gD}$', fontsize=12)
ax.set_ylabel('Normalized fountain height $h_{\{max\}}/D$', fontsize=12)
ax.set_title('Ring Fountain: Validation against Worthington Jet Scaling Law', fontsize=13)
ax.legend(loc='upper left', fontsize=10)
ax.grid(True, which='both', linestyle='--', alpha=0.5)

# 添加参数标注
textstr = f'Simulation parameters:\n' \
          f'$D$ = 20-60 mm, $d/D$ = 0.1\n' \
          f'$U$ = 1-4 m/s, $Fr$ = {Fr_sim.min():.1f}-{Fr_sim.max():.1f}\n' \

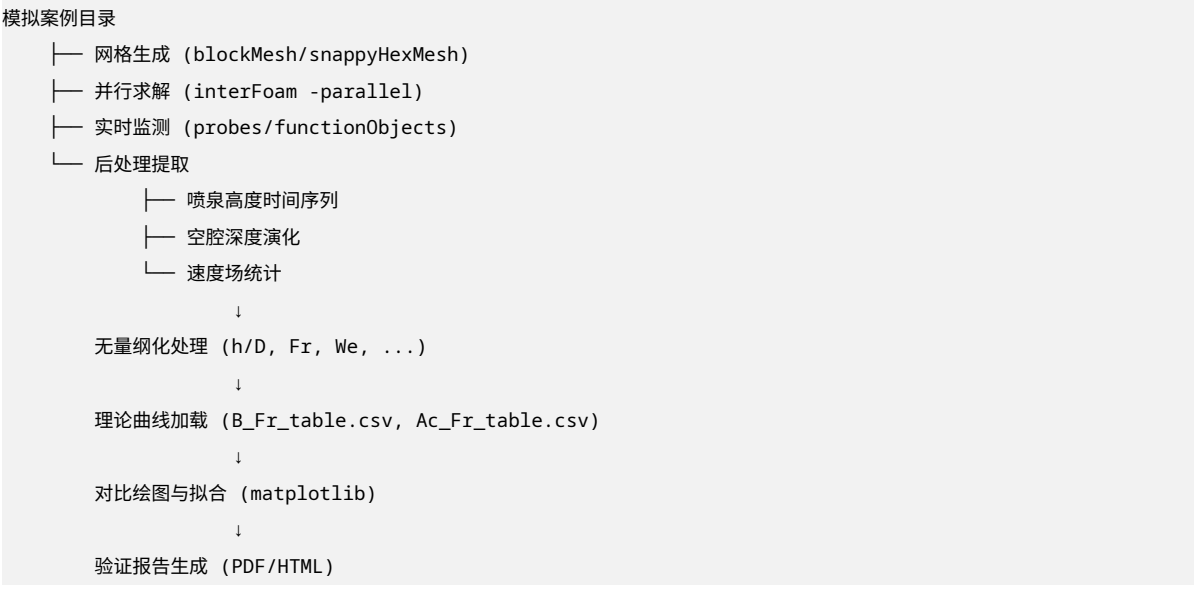
```

```
f'Grid: ~2M cells, $y^+ < 5$'
ax.text(0.02, 0.98, textstr, transform=ax.transAxes, fontsize=9,
        verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.savefig('fountain_validation_loglog.png', dpi=300, bbox_inches='tight')
plt.savefig('fountain_validation_loglog.pdf', bbox_inches='tight')
```

5.3 Python 验证脚本框架

5.3.1 数据管道设计 完整自动化流程:



5.3.2 补充验证：球体入水标度律 Jana et al. (2025) 的球体入水模型可作为几何相似性对比。尽管环与球的冲击机制不同（环形成轴对称空腔，球形成三维空腔），但在高 Froude 数极限下，射流高度的标度行为可能趋同。统一无量纲框架的构建尝试：

几何	特征长度	空腔类型	射流机制	标度指数 B
圆环	直径 D	轴对称环形	轴向汇聚	~0.8
球体	直径 D	三维半球形	轴向汇聚 + 径向收缩	~0.7-0.9

通过对比两种几何的模拟数据，可以评估标度律的普适性和几何特异性效应。

5.4 批处理与 workflow 优化

5.4.1 完整自动化脚本

```
#!/bin/bash
# run_ring_fountain_study.sh: 完整自动化 workflow

set -e # 错误即退出

STUDY_NAME="ring_fountain_Fr_scan_$(date +%Y%m%d)"
```

```
mkdir -p "$STUDY_NAME" && cd "$STUDY_NAME"

# 1. 生成基础案例模板
cat > make_base_case.sh << 'EOF'
# 创建标准圆环入水案例结构...
EOF
bash make_base_case.sh

# 2. 参数扫描
cat > param_scan.sh << 'EOF'
# 嵌套循环生成所有工况...
EOF
bash param_scan.sh

# 3. 监控与错误恢复
cat > monitor_jobs.sh << 'EOF'
# 检查运行状态，失败自动重提交...
EOF

# 4. 结果聚合与验证
python3 << 'PYEOF'
# 调用 aggregate_results.py + validation_plot.py
PYEOF

echo "Study complete: $STUDY_NAME"
```

5.4.2 计算资源管理

优化策略	实施方法	效果
并行规模匹配	每核心 5-10 万单元	最优负载均衡
I/O 瓶颈缓解	collated 格式 + 本地 SSD 缓存	减少 50%+ 写等待
检查点机制	writeControl adjustableRunTime	故障快速恢复
结果归档	自动 tar + 转移至冷存储	磁盘空间管理

6. 版本差异与兼容性说明

6.1 OpenFOAM Foundation v12 特性

特性	说明	影响
模块化求解器架构	foamRun + 求解器模块	interFoam 可由 incompressibleVoF 模块实现
非共形耦合 (NCC)	nonConformalProcessorCyclic	多区域网格自动处理
改进的 VOF 算法	优化 MULES 与界面压缩	稳定性提升，默认参数可能变化
函数对象位置调整	推荐 system/functions	更清晰的项目结构

6.2 与 ESI/OpenCFD 版本差异

方面	Foundation (openfoam.org)	ESI/OpenCFD (openfoam.com)
求解器命名	interFoam	interFoam / multiphaseInterFoam
物性文件组织	集中 transportProperties	分离 physicalProperties.phase
湍流模型关键字	momentumTransport (推荐)	turbulenceProperties
边界条件语法	标准	扩展关键字 (fixedMeanValue 等)
版本号	v12 (年度发布)	v2312 / v2406 (月份发布)

双向兼容性建议：使用标准字典语法，避免版本特定扩展；关键案例在两种版本上验证。

6.3 废弃特性与迁移指南

特性	状态	推荐替代
直接 sigma 数值	⚠ 已废弃 (仍兼容)	✅ 显式 surfaceTensionModel 子字典
fvSchemes 旧格式关键字	⚠ 计划移除	✅ 完整显式指定 (default none)
编译型自定义模型	⚠ 维护成本高	✅ coded 机制快速原型
controlDict 内嵌函数对象	⚠ 不推荐	✅ system/functions 独立文件